

Урок 22. Базы данных. Типы, виды. Создание БД. Часть 2.

Давайте рассмотрим ситуации, зачем мы реализовываем именно API + БД, а не просто БД, но перед этим три уточнения:

- **БД** – это хранилище.
- **API** – это **удобный и безопасный интерфейс** доступа к данным для разных программ/пользователей, в работу которого не всегда включена только работа с БД
- **Пользователь** – тот, кто использует API для рутинных задач, не разработчик, а простой обыватель, чаще это клиент, купивший программное обеспечение (например наше API) под свои бизнес задачи.

А теперь возможности API перед «голой» БД:

1) Доступ из разных мест

База (особенно не SQLite, а PostgreSQL) обычно живёт на сервере.

API — это “дверь” к данным, через которую могут работать:

- сайт (front-end)
- мобильное приложение
- другой сервис
- бот
- админка

Без API каждому пользователю пришлось бы напрямую подключаться к БД — это неудобно и неопасно для данных.

2) Безопасность

Если дать пользователю прямой доступ к БД:

- надо раздавать логины/пароли к базе
- пользователь может случайно/злонамеренно сделать вредный запрос
- сложно ограничить права

API позволяет:

- делать авторизацию (кто ты?)
- проверять права (что тебе можно?)
- фильтровать/ограничивать данные, которые отдаём

3) Правила и проверки (бизнес-логика)

БД просто хранит данные. А установленные правила работы вроде:

- “grade должен быть 1–11”
- “student_id уникальный”
- “нельзя удалить ученика, если он в расписании”
- “фамилия должна быть строкой”

Все это удобнее держать в API (или сервисном слое), чтобы **все кто пользуется API** соблюдали одинаковые правила.

4) Удобный формат и стабильный контракт

Пользователям нужны данные в **JSON** и в понятном виде.

API превращает таблицы и связи в то, что удобно использовать:

- /students?grade=3&last_name=Иванов → список JSON
- ошибки в виде кодов 400/404/409 + сообщение

Прямой SQL к БД не даёт такого “контракта” для фронта.

5) Независимость: можно менять БД, не ломая API пользователя

Если пользователь ходит напрямую в БД, они зависят от таблиц и SQL.

Если пользователи ходят в API, мы можем:

- поменять SQLite на PostgreSQL
- переименовать таблицы
- добавить индексы
- изменить хранение

Тем самым пользователям не нужно ничего менять, пока API-ответы те же.

Практическая работа:

Сегодня мы рассмотрим работу SQLite + **SQLAlchemy Core**, но уже на примере нашего API, который мы рассматривали ранее, пока что без добавления API.

1. Подключаем библиотеку:

```
import sqlalchemy as db
```

2. Первым делом нужно подключиться к базе данных и указать метаданные.

```
engine = db.create_engine("sqlite:///students.db",
future=True)
metadata = db.MetaData()
```

3. Создадим базу данных для хранения информации о наших учениках.

```
students = db.Table(
    "students", metadata,
    db.Column("student_id", db.Integer, primary_key=True),
    db.Column("first_name", db.Text, nullable=False),
    db.Column("last_name", db.Text, nullable=False),
    db.Column("date_of_birth", db.Text, nullable=False), #
    YYYY-MM-DD
    ...
```

Дописываем все поля самостоятельно.

4. Теперь, чтобы создать объект таблицы, вызовем функцию create_all:

```
metadata.create_all(engine)
```

5. Теперь нужно запустить движок и создать подключение:

```
with engine.begin() as conn:
```

Остальной код пишем внутри with.

6. INSERT: Добавляем новых учеников. По аналогии с прошлым уроком добавьте несколько учеников в созданную таблицу. Добавьте оставшиеся поля сами.

```
conn.execute(
    db.insert(students),
    [
        {
            "student_id": 1,
            "first_name": "Иван",
            "last_name": "Иванов",
            "date_of_birth": "2017-05-15",
            ...
        },
        ...
    ],
    ...
)
```

7. **SELECT all:** Выводим элементы базы данных в консоль. Чтобы вывести всех учеников по их id, используем метод `order_by` для сортировки номеров.

```
rows =
conn.execute(db.select(______).order_by(students.c.student_id)).fetchall()
print("ALL:", rows)
```

8. **SELECT by grade:** Фильтруем данные, например по 3 классу. Сделайте это самостоятельно и добавьте сортировку по фамилии: `.order_by(students.c.last_name)`

9. **SELECT by grade + last_name (без учёта регистра).**

```
grade_value = 3
last_name_value = "иванов" # в любом регистре
rows = conn.execute(
    db.select(students)
        .where(students.c.grade == grade_value)
        .where(db.func.lower(students.c.last_name) ==
db.func.lower(last_name_value))
        .order_by(students.c.student_id)
).fetchall()
print("GRADE 3 + last_name=иванов (case-insensitive):",
rows)
```

10. **UPDATE:** Обновляем данные, например, поменяем grade у student id=1 на 4. Пробуем самостоятельно.

```
conn.execute(
    db.update(____)
        .where(____)
        .values(____)
)
```

Делаем SELECT all, чтобы проверить, что изменение сработало.

11. **DELETE:** Удаляем данные, например, ученика с id=2. Пробуем самостоятельно.

```
conn.execute(
    db.delete(____).where(____)
)
```

Делаем SELECT all, чтобы проверить, что удаление сработало.

Запускаем, проверяем каждый метод.